

OSDBMS-V-2026-T038

PRODUCT REQUIREMENTS DOCUMENT

WALnut

A Concurrent Auction Engine with Deadlock Detection & Recovery

Team: The Acid Mechanics – T038

Vinayak Suyal · Sushmit Singh Rawat · Kanak Rawat · Suhavi Jugran

Version 1.0

Status: Draft for Review

Document Control

Version	Date	Author	Summary of Change
1.0	Week 1	The Acid Mechanics	Initial PRD derived from approved project proposal

Table of Contents

- 1. Introduction
- 2. Problem Statement & Motivation
- 3. Goals & Objectives
- 4. Scope
- 5. Users & Stakeholders
- 6. Functional Requirements
- 7. Non-Functional Requirements
- 8. Data Model
- 9. System Overview
- 10. Milestones & Timeline
- 11. Team & Responsibilities
- 12. Risks & Mitigations
- 13. Acceptance Criteria
- 14. References

1. Introduction

1.1 Purpose

This Product Requirements Document (PRD) defines what WALnut must do, for whom, and to what standard of correctness, before implementation begins. It translates the approved project proposal into concrete, testable requirements that the team can build against and that reviewers can use to evaluate deliverables at each weekly milestone.

1.2 Project Summary

WALnut is a lightweight, multithreaded auction data engine built from scratch in C++17. Rather than supporting arbitrary SQL, it executes a small, fixed set of parameterized operations — such as placing a bid or retrieving the current highest bid — over a custom page-based storage layer. Each bid is processed as an ACID transaction under Strict Two-Phase Locking (2PL). A wait-for graph detects cycles between transactions waiting on one another's locks and resolves them by rolling back a selected victim. Write-Ahead Logging (WAL) allows the engine to recover committed auction state after an abrupt crash. A React + TypeScript dashboard exposes both the bidding surface and a live systems-monitor view.

1.3 Definitions & Acronyms

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — the correctness guarantees for a transaction.
2PL / Strict 2PL	Two-Phase Locking; “Strict” means all exclusive locks are held until commit/rollback.
WAL	Write-Ahead Logging — changes are logged before the corresponding data page is flushed to disk.
Wait-for Graph	A graph of transactions where an edge $T_i \rightarrow T_j$ means T_i is waiting on a lock held by T_j ; used to detect deadlock cycles.
Victim	The transaction chosen for rollback to break a detected deadlock cycle.
Buffer Pool	An in-memory cache of on-disk pages, managed with a replacement policy (Clock/LRU).
SSE	Server-Sent Events — a one-way HTTP push channel used to stream live engine telemetry to the dashboard.
MVP	Minimum Viable Product — here, a persistent engine that safely processes concurrent bids (targeted end of Week 5).

2. Problem Statement & Motivation

Modern applications rely on databases to store and process large volumes of data safely, even when many users act on that data at the same time. WALnut is motivated by a concrete version of this problem: a live auction, where multiple bidders compete for the same item and the system must guarantee that (a) every accepted bid is

correct, (b) no two bids can both be recorded as the winning bid, and (c) a crash never loses or duplicates a committed bid.

Production engines such as PostgreSQL and SQLite already solve this class of problem, but they are highly complex and largely abstracted away from their users, which makes it hard to directly observe how locking, buffer management, storage, and recovery interact. WALnut is intentionally scoped as a small, educational storage and transaction engine — not a general-purpose DBMS — built around one fixed, observable workload, so every one of these mechanisms can be implemented, tested, and demonstrated within an eight-week project.

The project deliberately combines Operating Systems concepts (thread pools, synchronization primitives, buffer management, deadlock handling) with core DBMS concepts (schema design, indexing, transaction management, concurrency control, recovery) around this single workload.

3. Goals & Objectives

3.1 Project Objectives

- Build a persistent, page-based auction data engine supporting a fixed set of parameterized operations instead of general-purpose SQL.
- Guarantee safe multithreaded concurrency: simultaneous bids on the same item must never corrupt or duplicate auction state.
- Implement a custom storage manager and buffer pool for file, page, and memory management.
- Detect and resolve deadlocks using a wait-for graph instead of a resource-declaration scheme.
- Protect committed data through Write-Ahead Logging, checkpointing, and crash recovery.
- Provide a live dashboard where transactions, locks, the wait-for graph, WAL activity, and engine performance can be observed.

3.2 Success Metrics

Metric	Target
Concurrent correctness	Zero lost/duplicated bids under concurrent load in test suite
Deadlock resolution	100% of induced reserveBundle deadlock cycles detected and resolved without manual intervention
Crash recovery	100% of committed transactions survive simulated crash; no uncommitted transaction is applied
MVP delivery	Safely-concurrent placeBid engine functional by end of Week 5
Observability	Dashboard reflects lock table, wait-for graph, and WAL activity within ~1s of the underlying event
Performance baseline	Benchmark report covering latency, throughput, buffer hit rate, and deadlock frequency under load

4. Scope

4.1 In Scope — Core Requirements

- Fixed auction schema: Users, Items (with embedded current_bid / current_winner / status), Bids.
- Fixed operation API — no free-form SQL or query optimizer.
- Custom page-based file storage, written and read directly by the engine.
- Buffer pool with one replacement policy (Clock or LRU).
- One index type: hash index on item_id.
- POSIX thread pool, mutexes, condition variables.
- Strict Two-Phase Locking for every placeBid transaction.
- Wait-for graph, cycle detection, victim rollback and retry.
- Write-Ahead Logging with replay/undo recovery.
- Test scripts generating concurrent bids and a deliberate deadlock scenario.

4.2 Out of Scope — Stretch Goals

Pursued only if time remains after the core engine works end-to-end:

- B+ Tree index in place of the hash index.
- SCAN / C-SCAN disk-request scheduling simulation.
- Full checkpointing beyond the basic recovery path.
- Multi-item bundle transactions as a general feature, beyond the fixed deadlock-demo operation.
- A tiny fixed SQL subset (single-table SELECT/INSERT/UPDATE) layered on top of the operation API.
- Deployment of the engine and dashboard to a small persistent-disk VM for portfolio purposes.

4.3 Assumptions

- A single engine instance serves all requests; the system is not designed to scale horizontally across multiple processes or machines.
- Concurrent transactions may read and write the same item, which is precisely why synchronization and locking are required.
- The number of simultaneous scripted load-test bidders remains within the capacity of a single development/demonstration machine.
- Authentication and authorization are out of scope; a demo operator selects from a small set of fixed user_ids in the dashboard to place bids manually — no login flow is required to exercise the system live.
- “Simulated bidders” refers specifically to the automated scripts used for concurrency load and the deterministic reserveBundle deadlock scenario (FR-7); it does not mean bids can only be placed programmatically — manual bidding via the UI is a core requirement (FR-11), since the live demo depends on it.
- Catastrophic hardware failure (disk corruption) is out of scope; recovery targets process crashes and abrupt restarts.

4.4 Constraints

- Eight-week delivery window, aligned to the weekly milestone plan.
- Implementation language fixed to C++17 for the core engine; no third-party embedded database libraries.
- Team of four; each core subsystem has one primary owner (see Section 11).

5. Users & Stakeholders

User / Stakeholder	Description	Primary Needs
Bidder — Interactive (Demo)	A person operating the bidding UI directly — e.g. during the live demo — manually placing bids as a chosen fixed user_id.	A real, clickable bidding UI that submits an actual placeBid transaction and reflects the result immediately — not a read-only mock.
Bidder — Scripted (Load Test)	Automated test scripts issuing concurrent placeBid / reserveBundle calls to generate load and deterministically trigger the deadlock scenario.	Reliable concurrent execution over the same transaction path as manual bids, for stress-testing and grading evidence.
System Observer	Anyone viewing the systems-monitor dashboard during a demo or review.	Real-time visibility into locks, the wait-for graph, buffer pool, and WAL activity.
Course Evaluator	Faculty/reviewer assessing the project against OS + DBMS learning outcomes (OSDBMS-V-2026-T038).	Clear evidence that concurrency control, deadlock handling, and recovery are implemented correctly, not simulated.
Development Team	The Acid Mechanics (T038): 4 members, each owning a core subsystem.	A shared, unambiguous spec to build and integrate against.

6. Functional Requirements

Each functional requirement corresponds to one operation in the engine's fixed API. All operations are exposed internally and, from Week 8, over the embedded REST API.

ID	Requirement	Description	Priority
FR-1	createUser(user_id, username)	Register a new user identified by a fixed user_id.	Core
FR-2	createItem(item_id, title, starting_price)	Register a new auction item with an initial asking price.	Core
FR-3	placeBid(item_id, user_id, amount)	Execute as an ACID transaction: acquire exclusive lock on the item, validate amount > current highest bid, append bid record, update item's current_bid/current_winner, write WAL record, commit and release locks. Must be reachable identically from a manual UI click (live demo) and from a scripted load-test call — same code path, no demo-only shortcut.	Core

ID	Requirement	Description	Priority
FR-4	getHighestBid(item_id)	Return the current highest bid and winner for an item.	Core
FR-5	listBids(item_id)	Return the full bid history for an item.	Core
FR-6	closeAuction(item_id)	Mark an item's auction as closed; no further bids accepted.	Core
FR-7	reserveBundle([item_A, item_B], user_id)	Deliberately constructed multi-lock, all-or-nothing operation used to force and demonstrate a circular wait between two transactions requesting the same two items in mixed order.	Core (test scenario)
FR-8	Deadlock detection & resolution	Background detector periodically scans the wait-for graph for cycles; on detection, selects a victim transaction, rolls it back, releases its locks, and retries it automatically.	Core
FR-9	WAL logging & crash recovery	Every update is logged (before/after image) prior to page flush. On restart, committed transactions are redone and uncommitted ones undone.	Core
FR-10	Live monitoring dashboard	Bidding view (active items, highest bid, history) and systems-monitor view (lock table, wait-for graph, buffer-pool hit/miss rate, WAL activity), updated via SSE.	Core
FR-11	Interactive manual bidding (UI)	The bidding view's "Place Bid" control is a live, functional trigger — not decorative. A demo operator picks a fixed user_id from a small selector and submits a real placeBid, watching the highest bid, lock table, and (if applicable) wait-for graph update immediately. This is what the live demo runs on; scripted bidders (FR-7, load tests) are a separate, additional traffic source layered on the same engine.	Core

7. Non-Functional Requirements

Category	Requirement
Concurrency Safety	No interleaving of concurrent placeBid transactions may produce a lost update, dirty read, or duplicated winning bid. Enforced via Strict 2PL.
Reliability	Every committed bid must survive a process crash; every uncommitted transaction at the time of crash must be fully undone on recovery.
Deadlock Handling	The engine must never remain permanently deadlocked; a background detector must resolve any wait-for cycle within a bounded detection interval.
Performance	The engine must sustain the benchmark's concurrent bidding workload with measurable, reported latency and throughput; buffer pool hit rate must be observable under varying frame counts.

Category	Requirement
Observability	Internal state (locks held/waiting, wait-for graph edges, WAL writes, buffer hit/miss) must be exposed live to the dashboard, not just inferable from logs after the fact.
Testability	Concurrency and recovery behavior must be exercisable through repeatable, scripted test scenarios (including the reserveBundle deadlock demo and fault-injected crash recovery).
Maintainability	Core subsystems (storage, buffer pool, lock manager, deadlock detector, recovery manager, API) are decoupled enough that one owner can develop and test a subsystem with minimal blocking on others.
Portability	The system must build and run on a single standard development machine (Linux/macOS) using CMake; no distributed infrastructure is required.

8. Data Model

WALnut is built around one small, fixed relational schema, normalized to Third Normal Form. A full ER diagram and functional-dependency listing is maintained in the System Architecture Document; the table below summarizes the three relations.

Relation	Key Attributes
Users	user_id (PK), username
Items	item_id (PK), title, starting_price, status, current_bid, current_winner
Bids	bid_id (PK), item_id (FK → Items), user_id (FK → Users), amount, bid_time

Auction state (current_bid, current_winner, status) is embedded directly in Items rather than in a separate Auctions relation — sufficient to demonstrate keys, functional dependencies, and normalization without adding schema surface the demo does not need.

9. System Overview

WALnut consists of a client/frontend layer (React + TypeScript bidding UI and monitoring dashboard), an API layer (embedded REST server over HTTP/SSE), the fixed operation API, and a database engine core (transaction & lock manager, deadlock manager, index manager, buffer manager, recovery manager) sitting on a custom storage manager. Full component-level design, the high-level architecture diagram, and the technology stack are detailed in the companion System Architecture Document.

10. Milestones & Timeline

Week	Milestone	Deliverable
1	Design Freeze	Finalize fixed auction schema, ER diagram, functional dependencies, operation API, and architecture.

Week	Milestone	Deliverable
2	Storage Foundation	On-disk page format, file manager, record layout, database initialization.
3	Buffer & Indexing	Buffer pool (pin/unpin, dirty pages, Clock/LRU) and hash index on item_id.
4	Single-threaded Operations	Fixed auction operations wired end-to-end without concurrency.
5	Concurrency Core (MVP)	POSIX thread pool, lock manager, Strict 2PL; atomic placeBid. MVP: persistent engine safely processing concurrent bids.
6	Deadlock Handling	Wait-for graph, cycle detection, victim selection, rollback/retry; validated via reserveBundle.
7	Recovery	Write-Ahead Logging, commit/abort records, checkpointing, crash-recovery tests.
8	Integration & Demo	Embedded REST API + dashboard, concurrent-workload benchmarks, final documentation, demo script.

11. Team & Responsibilities

Member	Primary Responsibility
Vinayak Suyal (Lead)	Storage manager, page format, buffer pool, page replacement
Sushmit Singh Rawat	Transaction manager, lock manager, Strict 2PL, wait-for-graph deadlock detector
Kanak Rawat	Write-Ahead Logging, recovery, hash index, test workloads
Suhavi Jugran	Fixed operation API, embedded REST server, dashboard, performance visualization

Every member is expected to understand the complete architecture; the table above reflects primary ownership only, so integration responsibilities are clear from Week 1.

12. Risks & Mitigations

Risk	Impact	Mitigation
Race conditions in hand-rolled lock manager	Silent data corruption, hard to debug	Unit + stress tests from Week 5; deterministic reserveBundle scenario to validate wait-for graph early
Deadlock detector false negatives/positives	Engine hangs or unnecessary aborts	Bounded detection interval; scripted cyclic-wait test as acceptance gate for Week 6
WAL/recovery bugs losing or double-applying bids	Violates core correctness goal	Fault-injection tests at transaction and page-flush boundaries (Week 7)

Risk	Impact	Mitigation
Eight-week schedule slip	Core requirements incomplete by demo	Stretch goals explicitly deferred (Section 4.2); MVP checkpoint at Week 5 forces early integration
Buffer pool / storage manager integration bugs	Corrupted pages, crashes	Fixed page format frozen at Week 1 design freeze; incremental testing per week
Uneven subsystem coupling blocking parallel work	Team members blocked on each other	Clear primary ownership (Section 11) with stable interfaces between subsystems from Week 1

13. Acceptance Criteria

- By end of Week 5: engine persists data, accepts concurrent placeBid transactions via a thread pool, and Strict 2PL prevents any observed corruption or duplicate winning bid under concurrent test load. (MVP)
- By end of Week 6: a scripted reserveBundle scenario reliably produces a circular wait that the detector identifies, resolves via victim rollback, and retries — without manual intervention.
- By end of Week 7: a fault-injected crash during an in-flight transaction results in correct recovery — committed data intact, uncommitted data fully undone — verified by test script.
- By end of Week 8: embedded REST API and dashboard are live, showing bidding and systems-monitor views updating in real time; benchmark report (latency, throughput, buffer hit rate, deadlock frequency) and full documentation are complete.

14. References

- SQLite Documentation, “Architecture of SQLite.” <https://www.sqlite.org/arch.html>
- PostgreSQL Documentation, “Write-Ahead Logging (WAL).” <https://www.postgresql.org/docs/current/wal-intro.html>
- MySQL Documentation, “InnoDB Locking and Transaction Model.” <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-transaction-model.html>
- IBM Documentation, “POSIX Thread Programming.” <https://www.ibm.com/docs/en/zos/2.5.0?topic=services-thread-programming>
- Silberschatz, Korth, Sudarshan. Database System Concepts — Concurrency Control and Deadlock Handling.
- Silberschatz, Galvin, Gagne. Operating System Concepts — Deadlocks: Detection and Recovery.